Research and Development of an Automated Concept Drift Detection and Adaptive Machine Learning System

Master's Thesis

Student: Le Phuc Duc
Supervisor: Assoc.Prof.Dr. Thoai Nam

Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

2026

Presentation Outline

Motivation & Background

Real-world examples:

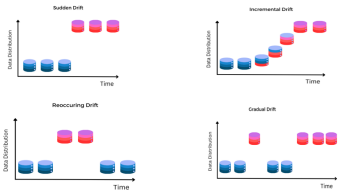
- Banking fraud detectors: false-alarm rate **triples** within a year of deployment.
- Predictive maintenance: model accuracy degrades as machine conditions change.

Underlying problem — Concept Drift:

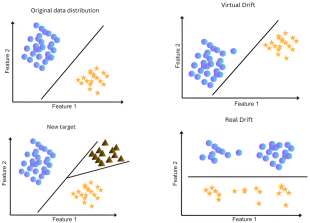
$$P_{\text{train}}(X, Y) \neq P_{\text{online},t}(X, Y)$$

Non-stationary environments

The mapping between inputs and outputs changes over time.
Example: legitimate transactions get flagged as fraud during a seasonal spike because the model's notion of "normal" is now stale.



Hình: Temporal drift patterns.



Hình: Distributional shift in the joint distribution.

Objectives & Contributions

Research Objectives

1. **Faster, more reliable detection:** integrate IDW-MMD into ShapeDD to raise throughput while keeping false alarms near zero.
2. **Unsupervised type identification:** build SE-CDT to categorize drift behaviors *without ground-truth labels*.
3. **End-to-end system:** deploy a closed-loop pipeline on Apache Kafka.

Three Main Contributions

1. **Detection (ShapeDD-IDW):** inverse density weighting + Gamma-null p-value $\rightarrow$ a calibrated test ($\alpha = 0.05$) with a modest end-to-end speedup over ShapeDD.
2. **Classification (SE-CDT):** an unsupervised geometric analyzer that labels 5 drift types directly from the MMD signal $\sigma(t)$.
3. **Adaptation:** a type-aware router that picks an update strategy (full reset, warm-start, model cache, ignore) based on the detected drift type.

Maximum Mean Discrepancy (MMD)

Idea: compare two distributions P, Q via their mean embeddings in an RKHS.

$$\mathrm{MMD}(P, Q) = \sup_{f \in \mathcal{F}} |\mathbb{E}_{X \sim P}[f(X)] - \mathbb{E}_{Y \sim Q}[f(Y)]|$$

Empirical estimator (Gaussian RBF kernel):

$$\widehat{\mathrm{MMD}}^2 = \underbrace{\tfrac{1}{n^2} \textstyle\sum_{i,j} k(x_i, x_j)}_{\text{within } P} + \underbrace{\tfrac{1}{m^2} \textstyle\sum_{p,q} k(y_p, y_q)}_{\text{within } Q} - \underbrace{\tfrac{2}{nm} \textstyle\sum_{i,p} k(x_i, y_p)}_{\text{cross } P-Q}$$

$$k(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma_k^2}\right) \quad \text{(Gaussian RBF)}$$

Role in drift detection

- $\widehat{\mathrm{MMD}}^2 \approx 0$: distributions look the same.
- $\widehat{\mathrm{MMD}}^2 \gg 0$: distributions disagree $\rightarrow$ **drift candidate**.

Why it suits this problem

Each sample is mapped into an RKHS via the kernel; the test compares the mean embeddings of the two windows.
Result: a single scalar test statistic, no distributional assumption, and no labels required at runtime.

ShapeDD — Shape-Based Drift Detection

Triangle Shape Theorem:

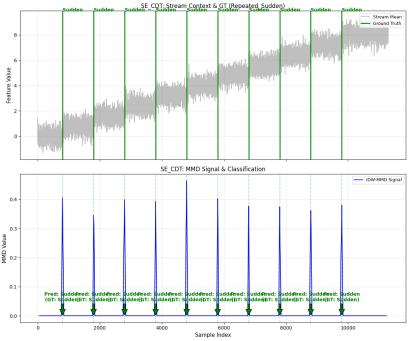
For a sudden drift at $t = 0$, the MMD trace satisfies

$$\sigma(t) = \underbrace{\|P - Q\|}_{\text{drift severity}} \cdot \underbrace{h_l(t)}_{\text{triangular envelope}} ,$$

with $h_l(t) = \max\left(0, 1 - \frac{|l-t|}{l}\right)$.

Processing pipeline:

1. Maintain a sliding window of $2l_1$ samples.
2. Build the kernel matrix K.
3. Compute MMD between the two halves.
4. Test significance with a permutation test.



Hình: Sudden drift produces the predicted triangular MMD trace.

Bottleneck

The permutation test re-builds the kernel thousands of times → heavy compute and the null distribution must still be estimated correctly.

Why a triangle?

As the sliding window crosses the change point, the proportion of post-drift samples grows linearly → MMD rises linearly until the window is fully on the new distribution, then falls back symmetrically.

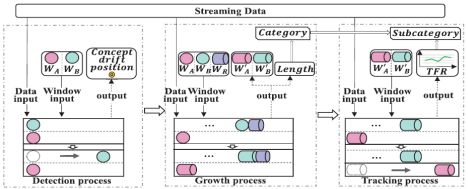
CDT-MSW — Supervised Classification Baseline

CDT-MSW (Guo et al., 2022) groups drift into:

- **TCD (Transient):** Sudden, Blip, Recurrent.
- **PCD (Progressive):** Gradual, Incremental.

Growth Process:

- Tracks the variance of accuracy degradation over a multi-sliding window.
- Short degradation $\rightarrow$ TCD; sustained degradation $\rightarrow$ PCD.



Hình: CDT-MSW architecture (label-dependent).

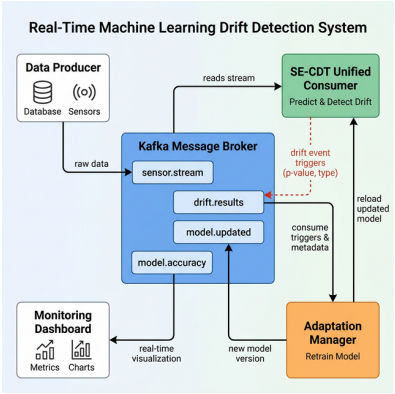
Limitation

CDT-MSW needs **ground-truth labels** at every step to compute the accuracy curve — impractical for IIoT streams where labels arrive late or never.

Motivation for SE-CDT

Replace the accuracy curve with the *shape* of the MMD signal $\sigma(t)$. The same TCD/PCD distinction can then be made **without any labels at runtime**.

SE-CDT System Architecture



Layer 1 — Detection
ShapeDD + IDW-MMD
Decides *when* a drift in $P(X)$ occurs.

Layer 2 — Classification
SE-CDT (geometric features + Concept Memory)
Decides *what kind* of drift it is (5 types).

Layer 3 — Adaptation
Type-aware strategy
Decides *how* the model is updated.

Contribution 1: Inverse Density-Weighted MMD

Limitation of standard MMD (uniform weights):

- Dense majority clusters dominate the discrepancy term.
- Early drifts that appear at distribution boundaries are washed out by the bulk.

Idea: weight by inverse local density

Down-weight points sitting in dense regions ($w_i \downarrow$); amplify isolated points near the boundary ($w_i \uparrow$). Boundary changes that signal drift get more influence on the test statistic.

IDW-MMD: weight w_i by inverse local density d_i.

Weighted MMD form

$$w_i \propto \frac{1}{\sqrt{d_i} + \epsilon}, \quad d_i = \sum_j k(x_i, x_j)$$

$$\widehat{\mathrm{MMD}}^2_{\mathrm{IDW}} = \sum_{i,j} w_i w_j \, k(x_i, x_j) + \sum_{p,q} v_p v_q \, k(y_p, y_q) - \frac{2}{nm} \sum_{i,p} k(x_i, y_p)$$

Statistical calibration: Gamma-null p-value

The permutation test is replaced by a Gamma-approximation p-value (Gretton et al., 2009). This avoids re-computing the kernel

Worked Example: 3-Point Window

A minimal observation window with 3 samples:

- Majority core: x_1, x_2 sit close together (normal behaviour).
- Boundary point: x_3 is isolated near the feature boundary (drift candidate).

1. Local density $d_i = \sum_j k(x_i, x_j)$

- Core: $d_1 = k(x_1, x_1) + k(x_1, x_2) \approx 1.9$
- Core: $d_2 = k(x_2, x_1) + k(x_2, x_2) \approx 1.9$
- Boundary: $d_3 = k(x_3, x_3) + \text{small} \approx 1.0$

2. Inverse weights $w_i = 1/\sqrt{d_i}$

- Core: $w_1 = w_2 = 1/\sqrt{1.9} \approx 0.72$ (down-weighted).
- Boundary: $w_3 = 1/\sqrt{1.0} = 1.0$ (kept at full weight).

Take-away

Standard MMD treats every point equally ($w_i = 1$). IDW gives the boundary point ($w_3 = 1.0$) more weight than the bulk ($w_{1,2} = 0.72$), so a small change at the boundary moves the test statistic. This is what gives early sensitivity to drift while keeping false positives low on stationary data.

Contribution 2: Unsupervised Shape Analysis of $\sigma(t)$

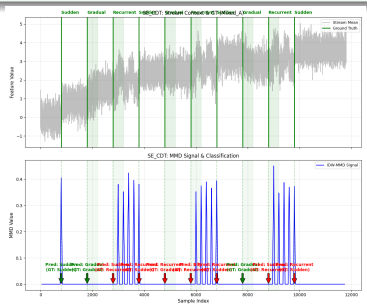
How can we type drift without ground-truth labels?

Read the *shape* of the MMD signal $\sigma(t)$ around the detection point. SE-CDT extracts **9 statistical features**:

- **Geometric (peak structure):**
  - n_p, WR, CV, SNR: count, width, periodicity, prominence.
  - PPR, DPAR: peak proximity and dual-peak amplitude ratio.
- **Temporal (behaviour over time):**
  - LTS, SDS, MS: trend strength, step score, monotonicity.

Why these features?

Each drift type has a typical signature:
- **Sudden:** a single sharp peak ($n_p = 1$, WR small, SNR high).
- **Gradual:** no clean peak, baseline rises into a plateau.
- **Incremental:** sustained upward trend (LTS high, MS high).
- **Blip:** a quick dual-peak ($n_p = 2$ with high amplitude ratio).



Hình: Sudden drift signature: $n_p = 1$, WR narrow.

SE-CDT Threshold Rules — Worked Example

Rule-based threshold classifier:

Condition	Label
$n_p \leq 3$, WR $<$ 0.15, SNR > 2	**Sudden**
$n_p = 2$, PPR $<$ 0.2, DPAR > 0.65	**Blip**
MMD match in Concept Memory	**Recurrent**
LTS $> 0.5 \vee$ MS > 0.6	**Incremental**
Otherwise	**Gradual**

Trace of one detection event:

Layer 1 has just fired a drift alert at time t^*.

1. Read the MMD trace $\sigma(t)$ over a window of $2l_1$ samples around t^*.
2. Count local maxima $\rightarrow n_p = 1$ (≤ 3).
3. Width at half maximum $\rightarrow$ WR $= 0.12$ (< 0.15).
4. Peak vs baseline noise $\rightarrow$ SNR $= 3.0$ (> 2).

$\Rightarrow$ First rule matches $\rightarrow$ label = **Sudden Drift**, which is then handed to the Adaptation layer.

Hybrid Pipeline: IDW-MMD to Detect, Standard MMD to Classify

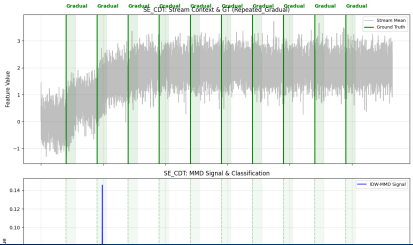
Why use two MMD variants instead of one?

- **IDW-MMD** is sharper and almost never fires on stationary windows — great as a gatekeeper, but its weighting also *flattens* the slow rise of a Gradual drift, removing the very shape SE-CDT needs.
- **Standard MMD** runs slower but *preserves the natural geometry* of $\sigma(t)$, including plateaus and trends — ideal as input to the shape classifier.

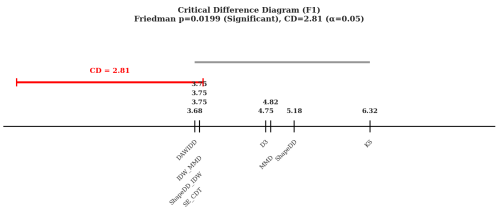
Pipeline

- **Layer 1 — Detection:** IDW-MMD runs continuously as the gatekeeper.
- **Layer 2 — Classification:** Standard MMD is computed *only at the detection time* t^* to feed SE-CDT.

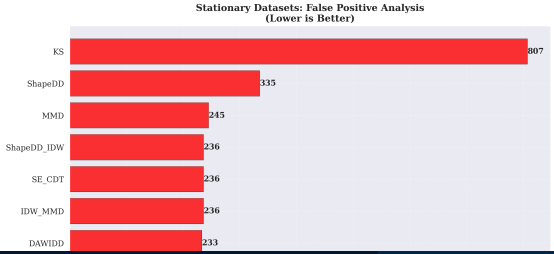
$\rightarrow$ Cheap to run continuously, accurate where it matters.



Result 1: Drift Detection ($\delta = 75$, cooldown $= 150$)



Hình: Critical Difference Diagram (Nemenyi, $\alpha = 0.05$)



Top methods (full 8-method table in the thesis):

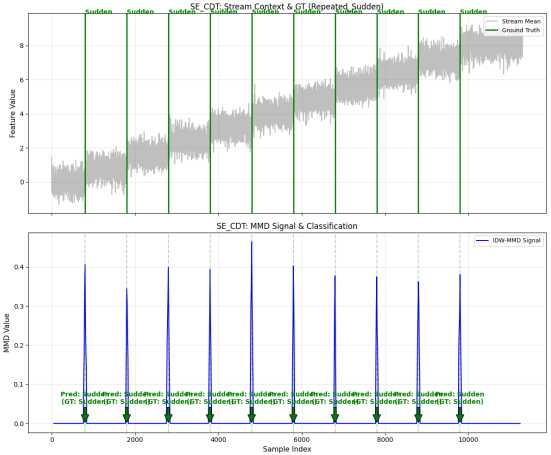
Method	F1	Prec.	FP/run
DAWIDD	**0.532**	0.436	10.3
IDW-MMD	0.531	0.432	9.6
SE-CDT	0.531	0.432	9.6
MMD (baseline)	0.524	0.426	10.6
D3	0.489	0.558	0.6
KS-Test	0.335	0.224	27.2

Protocol: TP if alert is within ± 75 samples of a true drift; cooldown $= 150$ suppresses duplicate alerts. IDW-MMD and SE-CDT share the same detection gate here, so their detection F1/FP are identical.

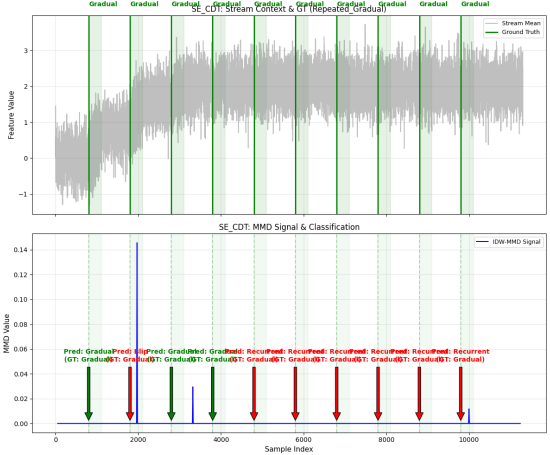
Key Results

- Empirical Type-I error on three stationary distributions: 0.025–0.075 for Standard MMD / IDW-MMD, 0.045–0.100 for the SE-CDT

Drift Signals at the Detector Output

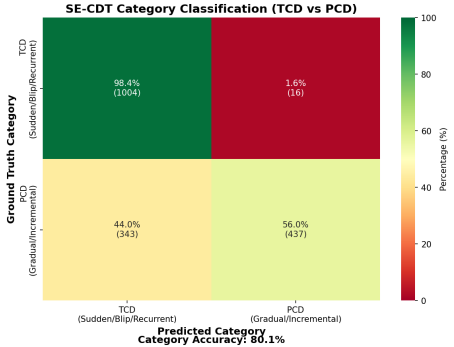


Hình: **Sudden drift:** sharp triangular peak — matches the Triangle Shape Theorem.



Hình: **Gradual drift:** IDW signal flattens (lower) while Standard MMD keeps the slope (upper) — motivates the hybrid pipeline.

Result 2: Unsupervised Classification (SE-CDT)



Hình: Category Matrix (TCD vs PCD) — CAT = **81.5%**

Two enhancements added

Concept Memory: ring buffer of post-drift snapshots, matched via Standard MMD. Recovers Recurrent classification when the same concept reappears.

Online Self-Calibration: rolling quantile baseline that adapts

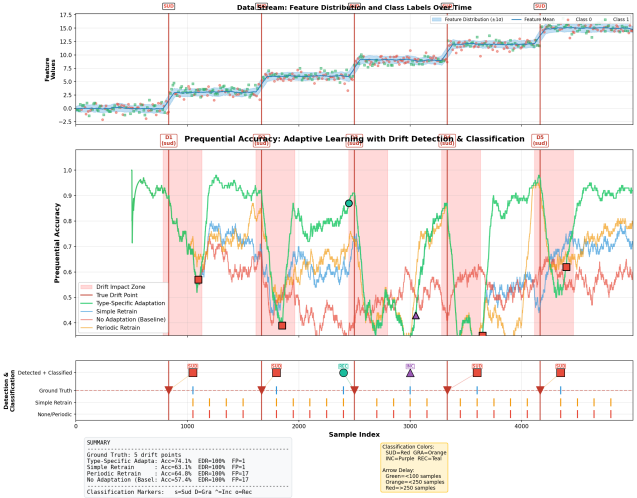
Compared with CDT-MSW (supervised baseline):

	SE-CDT	CDT-MSW	Note
Labels	**None**	Required	Unsupervised win
CAT	81.5%	87.0%	Close at −5.5pp
SUB	65.6% / 54.0%	86.7%	micro / macro
Recall	**92.6%**	82.5%	$1 - \overline{\text{MDR}}$ from paper Table 5

Per-type accuracy (oracle mode, 1800 events):

Drift Type	Acc.	Notes
Sudden	**82.4%**	Sharp, well-separated peaks
Recurrent	**89.0%**	Enabled by Concept Memory
Incremental	65.6%	Helped by temporal features
Gradual	32.9%	Window signal returns to baseline

Result 3: Model Adaptation (Prequential Accuracy, $N = 30$)



Strategy	Step.	Sudd.
Type-Specific	**71.4%**	**70.8%**
Periodic Retrain	64.0%	63.7%
Simple Retrain	63.0%	62.2%
No Adaptation	54.8%	53.7%

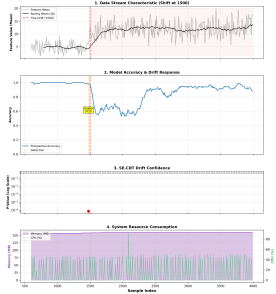
Why Type-Specific helps

A single "always reset" policy (Simple Retrain) discards still-useful history when the drift is gradual; a fixed schedule (Periodic) ignores the drift signal altogether.

Routing the update strategy by drift type — *Sudden* $\rightarrow$ *full reset*, *Gradual* $\rightarrow$ *warm-start* — adds **+16.6 pp** on Stepping and **+17.1 pp** on Sudden over No Adaptation, with Friedman $p < 10^{-15}$.

Hình: Stepping Drift: Type-Specific (blue) holds 71.4%, No Adaptation (red) drops to 54.8% as the distribution keeps moving away.

End-to-End Deployment on Apache Kafka



Detection

Drift alert fired at sample 1476, ahead of the planted change point at 1500.

Recovery

Model accuracy recovers from 59% to **93%** after the type-aware adaptation step.

Stability

Producer/detector/consumer run without back-pressure — closed-loop runs end-to-end.

Conclusion & Future Directions

Contribution Summary

1. **Detection:** ShapeDD-IDW with Gamma-null p-value → a calibrated test ($\alpha = 0.05$, H0-verified) with a modest end-to-end speedup over ShapeDD.
2. **Classification:** SE-CDT label-free, CAT = **81.5%**, SUB = **65.6%** (micro), EDR = **92.6%**.
3. **Concept Memory + Self-Calibration:** restore Recurrent classification and stabilise SNR/WR/CV thresholds online.
4. **Adaptation:** Type-Specific gains **+17.1 pp** on Sudden, **+16.6 pp** on Stepping vs No Adaptation ($N = 30$, Friedman $p < 10^{-15}$).
5. **System:** end-to-end Kafka closed-loop validated.

Limitations

- Gradual/Incremental still hard to separate — sliding-window MMD returns to baseline.
- Concept Memory threshold $\mu = 0.15$ chosen empirically from H0 noise floor.
- Adaptation: 3 scenarios, $N = 30$ runs (Friedman $p < 10^{-15}$).

Future Outlook

- Multi-scale reference window for Gradual/Incremental.
- Bayesian decision boundaries (confidence scores).
- Concept Memory v2 with learned threshold.
- Real-world IoT validation.

THANK YOU FOR LISTENING!

Q & A

Student: Le Phuc Duc — Supervisor: Assoc.Prof.Dr. Thoai Nam

Backup: IDW-MMD vs Optimally-Weighted MMD (Bharti et al.)

Question: "How is IDW-MMD different from the Optimally-Weighted MMD of Bharti et al. (2023)?"

Bharti et al. (OW-MMD)

- Domain: **likelihood-free inference** (ABC).
- Weights chosen by optimising against a target posterior.
- Requires solving a constrained convex optimisation per call.
- Not designed for streaming drift detection.

IDW-MMD (this work)

- Domain: **drift detection** in fast data streams.
- Weights from a closed-form heuristic: $w_i \propto 1/\sqrt{d_i + \epsilon}$.
- Cost: $O(n^2)$ for the kernel matrix, $O(n)$ extra for the weights — no optimisation step.
- Same *weighted MMD* family, different objective and very different runtime.

Summary: both are weighted MMD variants, but they target different problems (ABC vs streaming drift), use different weight functions, and have very different runtime profiles.

Backup: "F1 close to MMD baseline — where is the contribution?"

Question: "SE-CDT's F1 (0.531) is only marginally above MMD (0.524). Where is the gain?"

F1 alone hides the trade-off:

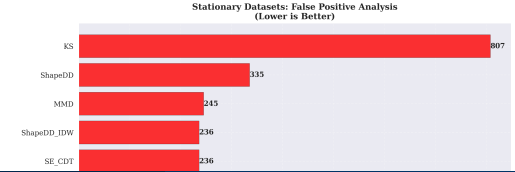
	SE-CDT	MMD	KS
F1	0.531	0.524	0.335
FP	**9.6**	10.6	27.2
Recall	0.737	0.732	0.775
Precision	0.432	0.426	0.224

In a production setting:

- Frequent FPs → **alert fatigue**, lost trust in the monitor.
- Investigating one false alarm typically costs more than missing one event.
- A miscalibrated permutation-style test inflates either FPs or misses.

What SE-CDT actually adds:

1. **Test calibrated near** $\alpha = 0.05$ — empirical Type-I error in the 0.025–0.100 band across detectors and three stationary distributions.
2. **Classification module on top of detection** — gives a drift type, not just a flag.
3. **Modest runtime speedup over ShapeDD** thanks to the Gamma-null p-value (no kernel re-build per shuffle).
4. Stays **fully unsupervised** at runtime.



Backup: Why does Concept Memory matter for Recurrent drift?

Question: "Recurrent drift requires concept matching — how does Concept Memory provide it?"

The detection challenge:

- A Recurrent stream is a sequence of *repeated* concept switches; each switch produces an MMD peak that, in isolation, looks like a Sudden drift.
- Without memory of past concepts, repeated visits are scored independently → Recurrent collapses into the Sudden branch.

How Concept Memory resolves it:

1. During every *stable period*, take a snapshot of the current data window and store it in a ring buffer of size $K = 8$.
2. When a new drift fires, compute Standard MMD between the new post-drift window and every snapshot.
3. If the smallest MMD $< \mu = 0.15$ → override the

Why $\mu = 0.15$?

$\mu = 0.05$ falls inside the MMD noise floor (false matches between independent stationary windows). An empirical sweep on H0 calibration data shows $\mu = 0.15$ separates "revisited concept" from "new concept" reliably.

Remaining limitations

Two failure modes remain on the harder Recurrent variants:

- Recurrent events that are *too far apart* push the original snapshot out of the ring buffer.
- Repeated Gradual streams produce flattened MMD signals that no snapshot matches well.

Backup: "CDT-MSW has higher CAT and SUB — is SE-CDT worse?"

Question: "CDT-MSW reports CAT 87.0% and SUB 86.7%, both higher than SE-CDT. Why frame SE-CDT as competitive?"

Side-by-side numbers

	SE-CDT	CDT-MSW
Labels	**None**	Required
CAT	81.5%	87.0%
SUB	65.6%	86.7%
Recall	**92.6%**	82.5%
Effective	**75.5%**	71.8%

Effective = CAT $\times$

Recall (probability of seeing a drift *and* typing it correctly).

CDT-MSW Recall $= 1 - \overline{\text{MDR}}$ from paper Table 5.

Reading the numbers in context:

- CDT-MSW reports CAT and SUB *conditionally* on drifts it detects; with paper recall $\approx 82.5\%$ it still misses $\sim 17\%$ of events.
- SE-CDT detects 92.6% of drifts and types every one of them without any labels.
- End-to-end, SE-CDT lands ~ 3.7pp above CDT-MSW on effective accuracy (75.5% vs 71.8%); the contribution is reaching this on **unsupervised** $P(X)$.

Trade-off

CDT-MSW is more accurate per detected event because it has access to ground truth labels. SE-CDT

Backup: "Why not deep learning?"

Question: "Couldn't a deep learning model solve this more accurately?"

Deep models for drift detection — limitations

- Need a large labelled dataset to train — the very thing we don't have on a streaming IIoT pipeline.
- Higher inference latency per window than a kernel computation.
- No closed-form theoretical bound on the test statistic.
- Decision is hard to explain to a domain expert.

Why MMD / ShapeDD fits this problem

- Fully unsupervised — no training run before deployment.
- Theoretical guarantees: Triangle Shape Theorem, Type-I error from H0 calibration.
- Decision rule is human-readable (a small tree of geometric features).
- One mechanism works across $P(X)$ shifts of arbitrary type.

Future direction: keep the kernel framework but plug in a learned feature extractor for high-dimensional inputs (Chapter 6).

Backup: behaviour on mild drift

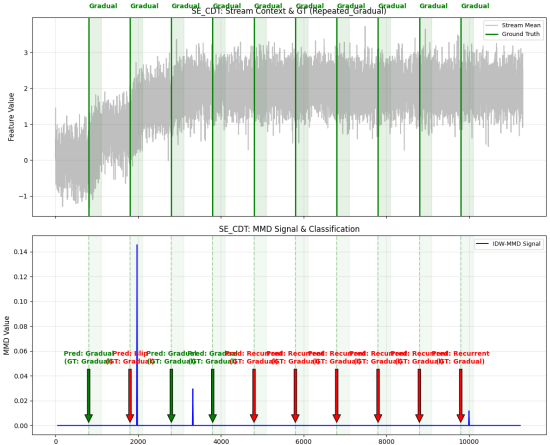
Question: "On the mild-drift scenario (Random Mild, $\delta = 0.125$) SE-CDT scored F1 $= 0.511$ vs MMD $= 0.563$ — where does the gap come from?"

Cause:

- IDW down-weights points sitting in dense (stable) regions.
- Mild drift, by definition, lives mostly inside those dense regions, so the IDW peak is shorter than Standard MMD's on the same window.
- Net effect: a few candidate peaks fall under threshold, so recall drops while precision is unchanged.

This is the intended trade-off:

- IDW-MMD biases toward fewer false alarms (FP ~ 9.6/run vs ShapeDD original 13.2, Standard MMD 10.6).



Backup: Permutation test vs Gamma-null p-value

Question: "Removing the permutation test — is the Gamma-null p-value still trustworthy?"

Permutation test (original ShapeDD)

- Reshuffles indices many times per window.
- Empirical null, no distributional assumption.
- Each shuffle re-builds the kernel matrix → heavy compute.

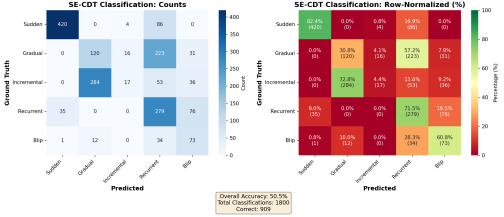
Gamma-null p-value (IDW-MMD)

- Moment-matched Gamma fit to the null MMD distribution (Gretton et al. 2009).
- Estimates the first two moments from a small number of index shuffles, reusing the kernel matrix.
- Complexity $O(n^2)$ for the kernel plus a few cheap shuffles.

Verification: The detection F_1 remains comparable to the permutation baseline; the H0-calibration on three stationary distributions reports an empirical Type-I error of 0.025–0.100 across detectors (Standard MMD and IDW-MMD inside the 95% CI of $\alpha = 0.05$, the SE-CDT composite slightly above on two distributions). End-to-end runtime is slightly faster than the ShapeDD baseline.

Backup: “Subcategory accuracy 65.6% — is it close to random?”

Question: “With 5 classes, 65% sounds close to random guessing.”



Hình: 5-class confusion matrix (SE-CDT)

It is well above random:

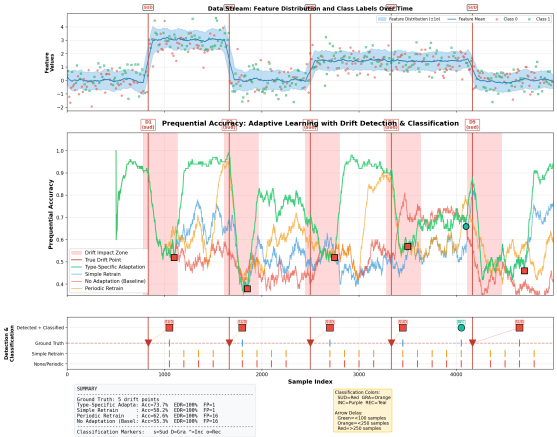
- Random baseline across 5 classes is 20%.
- Micro SUB = 65.6% is $\sim 3.3\times$ random and well above the 95% CI of a random predictor ($\sim 23\%$).
- Performance is **very uneven across classes**:

Drift Type	Accuracy
Recurrent	**89.0%**
Sudden	**82.4%**
Incremental	65.6%
Gradual	32.9%
Blip	0.0%

Why the gap? Recurrent benefits from Concept Memory; Sudden has the cleanest peaks. Incremental and Gradual look similar at the

Backup: "Is the adaptation gap statistically significant?"

Question: "Is Type-Specific really better, or is it run-to-run noise?"



Hình: Prequential accuracy on Sudden drift, $N = 30$ runs.

Friedman test (paired by seed):

- Stepping: $\chi^2 = 78.12$, $p = 7.8 \times 10^{-17}$
- Sudden: $\chi^2 = 86.76$, $p = 1.1 \times 10^{-18}$
- Mixed: $\chi^2 = 76.32$, $p = 1.9 \times 10^{-16}$

$p \ll 0.05$ on all three scenarios; the ranking among 4 strategies is not random.

Remaining limits

- Only 3 scenarios (Stepping, Sudden, Mixed).
- No Gradual/Incremental adaptation scenario yet.
- No real-world dataset with verified drift labels.

Listed in **Chapter 6: Conclusion and Future Directions**